

Formulating Markov Decision Processes

States, stochastic transitions, rewards, policies, and recursive value

Learning objectives

By the end of this lesson, you should be able to:

- 01 — Define the MDP tuple $\langle S, A, P, R, \gamma \rangle$.
- 02 — Test whether a state representation satisfies the Markov property.
- 03 — Distinguish trajectory, reward, return, and discount.
- 04 — Define policy π , V^π , and Q^π .
- 05 — Derive the Bellman expectation equation.
- 06 — Compute a fixed-policy value in a small MDP.

An MDP is a persistent stochastic decision model

The agent commits to a policy that maps states to action distributions across an extended trajectory.

Policy

A policy specifies behavior at every reachable decision state.

- Transitions are probabilistic, not adversarial.
- Rewards arrive along a trajectory.
- Value aggregates present and discounted future reward.

The MDP tuple defines dynamics and preference

$$\text{MDP} = \langle S, A, P(s'|s,a), R(s,a,s'), \gamma \rangle$$

- S is the state set.
- A is the action set.
- P gives successor probabilities.
- R gives transition reward; $0 \leq \gamma < 1$ discounts delay.

ENGINEERING IMPLICATION

Every transition distribution must be nonnegative and sum to 1.

The Markov property is a sufficiency claim

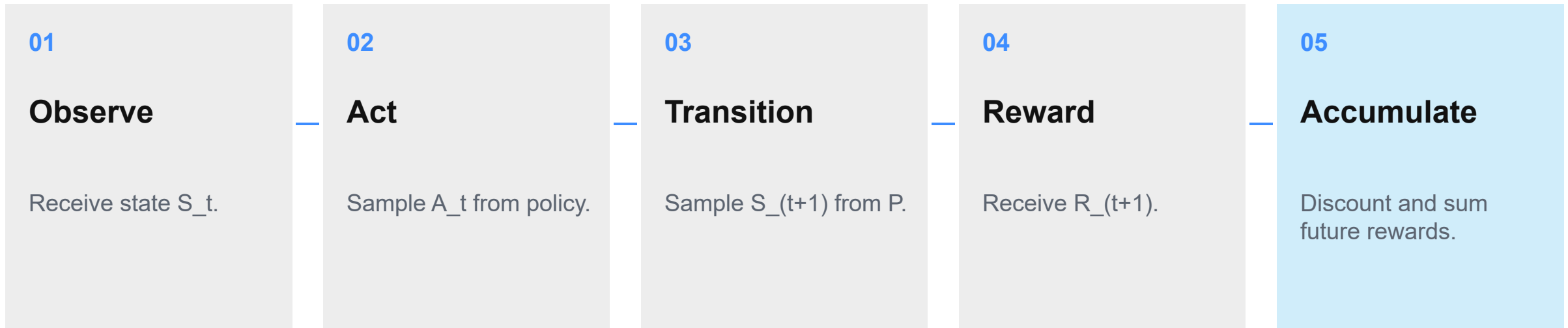
Markov state

- Predicts next outcome from current state and action.
- No extra history is required.
- Supports recursive value equations.
- Can include memory summaries.

Incomplete state

- Hidden history changes transitions.
- Same stored state has different futures.
- Bellman backup mixes incompatible cases.
- Add the missing variable or belief.

A trajectory generates discounted return



Return evaluates a sequence, while reward labels one transition.

A simulator exposes the MDP interface

```
state = env.reset()
done = false
return_value = 0
discount = 1
while not done:
    action = policy.sample(state)
    next_state, reward, done = env.step(action)
    return_value += discount * reward
    discount *= gamma
    state = next_state
```

Model contracts

- State is Markov.
- Policy sees state only.
- Step samples P .
- Terminal stops future reward.

Worked return calculation

SCENARIO

A trajectory yields rewards 2, -1, and 5 with $\gamma=0.9$.

- Weight the first reward by 1.
- Weight the second by 0.9.
- Weight the third by 0.9^2 .
- Add $2 - 0.9 + 4.05$.

The discounted return is 5.15.

Value functions answer related conditional questions

$V^{\pi}(s)$

State value

- Expected return from s .
- Follow π immediately.
- Averages over policy actions.
- One value per state.

$Q^{\pi}(s,a)$

Action value

- Expected return after action a .
- Then follow π .
- Supports action comparison.
- One value per state-action.

Policy $\pi(a|s)$

Behavior

- Action distribution.
- Can be deterministic.
- Defines on-policy values.
- Must normalize by state.

Checkpoint: repair a non-Markov state

CHECKPOINT

A machine's failure probability depends on how many consecutive hours it has run, but state stores only current temperature. Is the state Markov?

01

Identify hidden history that affects P .

02

Name a sufficient additional variable.

03

Explain the Bellman consequence.

Expectimax and MDPs share expectation but differ in scope

One game-tree decision

- Choose a root move.
- Chance may appear at nodes.
- Finite lookahead is common.
- No stationary behavior required.

MDP policy problem

- Choose behavior for all states.
- Environment persists.
- Long or infinite horizon.
- Recursive values seek consistency.

Bellman expectation decomposes one step

$$V^{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V^{\pi}(s')]$$

- Outer sum averages policy actions.
- Inner sum averages successor states.
- R is immediate reward.
- γV is discounted continuation value.

ENGINEERING IMPLICATION

A value function is a fixed point consistent with its policy and model.

Fixed-policy value in a two-state MDP

SCENARIO

In state A, a fixed action gives reward 1 and stays in A with probability 0.5 or terminates with probability 0.5; $\gamma=0.8$.

- Write $V(A)=1+0.8[0.5 V(A)+0.5*0]$.
- Collect $0.4 V(A)$ on the left.
- Obtain $0.6 V(A)=1$.
- Divide by 0.6.

$V(A)=5/3$, approximately 1.667.

Discounting has mathematical and modeling roles

gamma=0

Immediate reward only

- Future is ignored.
- Myopic behavior.
- Simple backup.
- Useful boundary test.

0<gamma<1

Geometric weighting

- Delayed reward matters.
- Bounded infinite return.
- Contraction later applies.
- Scale is interpretable.

gamma near 1

Long horizon

- Future decays slowly.
- Values can be large.
- Convergence can slow.
- Numerical tolerance matters.

Checkpoint: apply the Bellman equation

CHECKPOINT

A deterministic policy receives reward 3, moves to state B, and $V^{\pi}(B)=4$ with $\gamma=0.5$. What is $V^{\pi}(A)$?

01

Use the one available action.

02

Use successor probability 1.

03

Add reward and discounted continuation.

Validate an MDP before solving it

Transition normalization, terminal semantics, reward timing, and state sufficiency are executable preconditions.

- Assert each transition row sums to 1 within tolerance.
- Define terminal actions and continuation value explicitly.
- Use small hand-computed policies as numerical oracles.

Model

A solver cannot recover a guarantee from an inconsistent environment definition.

Key takeaways

- 01 — An MDP combines Markov state, actions, transition probabilities, rewards, and discount.
- 02 — A policy defines behavior over all states.
- 03 — V and Q are expected discounted returns.
- 04 — Bellman expectation is a recursive fixed-point condition for a policy.

Next: Value iteration and policy iteration replace evaluation of a fixed policy with optimal control.

Check your understanding

Q1

Classify a state as Markov or incomplete.

Q2

Compute a three-reward discounted return.

Q3

Distinguish V^{π} from Q^{π} .

Q4

Derive a two-state Bellman equation.

Q5

List MDP validation checks.